

QuickBite: Line-by-Line Code Walkthrough

This document explains **every file of code** in the project, in the order `run_all.sh` runs them. Line numbers match the files exactly, so keep the `.sql` file open beside this one.

When many lines follow the same pattern (for example 50 rows of sample data), the pattern is explained once, using one row as the example.

#	File	Lines	What it does
1	run_all.sh	37	Builds the whole database and saves the outputs
2	sql/01_schema.sql	174	Creates the tables, keys, constraints, trigger and view
3	sql/02_indexes.sql	28	Creates the search index and the FK indexes
4	sql/03_seed_data.sql	308	Inserts the sample data
5	sql/04_queries.sql	241	The 12 report queries
6	sql/05_index_demo.sql	67	Proves the index is faster
7	sql/06_integrity_tests.sql	46	Proves bad data is rejected

Notation used below: `--` starts an SQL comment. `#` starts a shell

comment. *PK* = primary key, *FK* = foreign key.

1. run_all.sh

A Bash script that rebuilds everything with one command.

Line	Code	Explanation
1	<code>#!/usr/bin/env bash</code>	The "shebang". It tells the OS to run this file with Bash, found through <code>env</code> so it works on any machine.
2–3	<code># Builds the QuickBite...</code>	Comments: what the script does and how to run it.
4	<code>set -euo pipefail</code>	Safety switches. <code>-e</code> stops at the first failing command. <code>-u</code> makes an undefined variable an error. <code>o pipefail</code> makes a pipeline fail if <i>any</i> part of it fails, not only the last.
6	<code>DB="\${DB:-quickbite}"</code>	The database name. Uses the environment variable if you set one, otherwise <code>quickbite</code> . That's why <code>DB=mydb ./run_all.sh</code> works.
7	<code>cd "\$(dirname "\$0")"</code>	Moves into the folder the script lives in, so the relative paths like <code>sql/...</code> work from anywhere.
8	<code>mkdir -p outputs</code>	Creates <code>outputs/</code> . <code>-p</code> means no error if it already exists.

10	<pre>PSQL=(psql -X -q -v ON_ERROR_STOP=1 -d "\$DB" -P pager=off)</pre>	Stores the psql command, with options, in an array so it's typed only once. <code>-X</code> : ignore your personal <code>~/.psqlrc</code> . <code>-q</code> : quiet. <code>ON_ERROR_STOP=1</code> : abort the file on the first SQL error, instead of carrying on. <code>-d</code> : which database. <code>pager=off</code> : print everything, never open less.
12	<pre>echo "==> Recreating..."</pre>	Progress message.
13	<pre>dropdb --if-exists "\$DB"</pre>	Deletes the old database, if it exists, so every run starts clean. △ This erases anything in a database of the same name.
14	<pre>createdb "\$DB"</pre>	Creates a fresh, empty database.
16	<pre>"\${PSQL[@]}" -f sql/01_schema.sql</pre>	Runs the schema file. " <code>\${PSQL[@]}</code> " expands the array from line 10.
17	<pre>... -f sql/02_indexes.sql</pre>	Creates the indexes.
18	<pre>... -f sql/03_seed_data.sql > /dev/null</pre>	Loads the data. Its output (the <code>setval</code> numbers) is thrown away into <code>/dev/null</code> .
20–28	<pre>"\${PSQL[@]}" -c "SELECT 'restaurant' ... UNION ALL ..."</pre>	One query that counts the rows in every table. <code>UNION ALL</code> stacks the seven <code>COUNT(*)</code> results from one table. tee outputs/00_row_counts.sql shows the result on screen and saves it to a file.

30–35	<pre>... -f sql/04_queries.sql > outputs/04_queries_output.txt 2>&1</pre>	Runs the queries, the index d and the integrity tests. Each o output is saved to a file. 2>&1 also captures error and notice messages (stderr), which is where the integrity test results are printed.
37	<pre>echo "Done."</pre>	Finished.

2. sql/01_schema.sql

Header and clean start (lines 1–10)

Line	Explanation
1–5	Header comment: project name, file purpose, normal form reached.
7	SET client_min_messages = warning; hides harmless NOTICE messages, such as "table does not exist, skipping" on the first run.
9–10	DROP TABLE IF EXISTS ... CASCADE; deletes all 7 tables if they exist, so the file can be re-run safely. IF EXISTS avoids an error on the first run. CASCADE also drops anything that depends on them: the view, the trigger, the FKs.

restaurant (lines 12–27)

Line	Code	Explanation
------	------	-------------

12–16	comment	Explains the 2NF fix. Cuisine and address now live here once , instead of on every menu row.
17	CREATE TABLE restaurant (	Starts the table definition.
18	restaurant_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	The PK. IDENTITY auto-numbers new rows (1, 2, 3...). BY DEFAULT still lets the seed file insert explicit IDs. PRIMARY KEY = unique + not null.
19	name VARCHAR(100) NOT NULL	Text of up to 100 characters. NOT NULL = required.
20	cuisine VARCHAR(40) NOT NULL	e.g. 'North Indian'. The first column of the search index.
21	location VARCHAR(60) NOT NULL	Area/locality, e.g. 'Kharghar'. The second column of the search index.
22	address VARCHAR(200) NOT NULL	Full street address. Kept separate from location, because you search by area, not by street.

23	<code>phone VARCHAR(15) NOT NULL UNIQUE</code>	Stored as text, not a number, because phone numbers can start with 0 and are never used in arithmetic. <code>UNIQUE</code> means no two restaurants share a number.
24	<code>rating NUMERIC(2,1) CHECK (rating BETWEEN 0 AND 5)</code>	An exact decimal with 2 digits in total, 1 after the point (e.g. 4.5). <code>CHECK</code> rejects ratings outside 0–5. It's nullable, because a new restaurant has no rating yet.
25	<code>is_active BOOLEAN NOT NULL DEFAULT TRUE</code>	Lets you hide a restaurant without deleting it. Defaults to active.
26	<code>CONSTRAINT uq_restaurant_name_location UNIQUE (name, location)</code>	A named composite unique constraint. The same chain can exist in two areas, but there can't be two branches with the same name in one area.
27	<code>);</code>	Ends the table.

`menu_item` (lines 29–45)

Line	Code	Explanation
------	------	-------------

30	comment	This table implements Restaurant 1 : Menuitem.
33	menu_item_id ... PRIMARY KEY	Auto-numbered PK, same pattern as above.
34–36	restaurant_id INTEGER NOT NULL REFERENCES restaurant(restaurant_id) ON UPDATE CASCADE ON DELETE RESTRICT	The FK. Every dish must belong to a real restaurant. ON UPDATE CASCADE: if a restaurant's ID ever changes, the dishes follow. ON DELETE RESTRICT: you can't delete a restaurant that still has dishes.
37	name VARCHAR(100) NOT NULL	The dish name.
38–40	category ... CHECK (category IN ('Starter', ..., 'Combo'))	Only allows the 8 listed categories. This works like a mini enum.
41	price NUMERIC(8,2) NOT NULL CHECK (price > 0)	Money is always NUMERIC, never FLOAT, because floats have rounding errors. 8 digits in total, 2 after the point (up to 999999.99). The price must be positive.

42	<code>is_veg</code> BOOLEAN NOT NULL	Veg or non-veg. No default, because it must be stated explicitly.
43	<code>is_available</code> BOOLEAN NOT NULL DEFAULT TRUE	A sold-out flag.
44	CONSTRAINT <code>uq_menu_item_per_restaurant</code> UNIQUE (<code>restaurant_id</code> , <code>name</code>)	One restaurant can't list "Masala Dosa" twice. Two <i>different</i> restaurants can.

customer (lines 47–58)

Line	Explanation
51	Auto-numbered PK <code>customer_id</code> .
52	<code>full_name</code> , required.
53	<code>email</code> ... NOT NULL UNIQUE. One account per email. This makes email a candidate key , which matters for the BCNF check.
54	<code>phone</code> ... NOT NULL UNIQUE. Also a candidate key.
55–56	<code>address</code> and <code>location</code> (area).
57	<code>joined_on</code> DATE NOT NULL DEFAULT CURRENT_DATE. If no date is given, today's date is filled in automatically.

delivery_agent (lines 60–72)

Line	Explanation
64	Auto-numbered PK <code>delivery_agent_id</code> .

65– 66	<code>full_name</code> is required. <code>phone</code> is required and unique.
67– 68	<code>vehicle_type</code> is limited by CHECK to Bicycle / Scooter / Motorbike / EV Scooter.
69	<code>vehicle_no</code> VARCHAR(15) UNIQUE is nullable , because a bicycle has no registration number. UNIQUE still allows many NULLs, since in SQL one NULL never equals another.
70	<code>home_zone</code> , the area the agent usually covers.
71	<code>is_active</code> flag, defaulting to TRUE.

food_order (lines 74–103)

Line	Code	Explanation
74– 81	comment	This one table implements three 1 : M relationships . Because all three FKs are NOT NULL, <i>every order ties to a valid restaurant, customer and delivery agent</i> . That's one of the outcomes the brief requires.
83	<code>order_id</code> ... PRIMARY KEY	Auto-numbered PK.
84– 86	<code>customer_id</code> INTEGER NOT NULL REFERENCES <code>customer</code> ... ON DELETE RESTRICT	Who placed the order. RESTRICT means you can't delete a customer who has order history.
87– 89	<code>restaurant_id</code> ... REFERENCES <code>restaurant</code> ...	Which restaurant it was ordered from.
90– 92	<code>delivery_agent_id</code> ... REFERENCES <code>delivery_agent</code> ...	Who delivers it.

93	<code>order_time</code> <code>TIMESTAMP</code> <code>NOT NULL</code> <code>DEFAULT CURRENT_TIMESTAMP</code>	Date and time of the order. Filled in automatically if not given.
94	<code>delivery_address</code> <code>VARCHAR(200)</code> <code>NOT NULL</code>	A snapshot of where this order went. It isn't a redundant copy of <code>customer.address</code> , because if the customer moves later, the old order must still show the old address.
95–97	<code>status</code> ... <code>DEFAULT 'Placed'</code> <code>CHECK (status IN (...))</code>	The order lifecycle: Placed → Preparing → Out for Delivery → Delivered, or Cancelled.
98	<code>delivered_time</code> <code>TIMESTAMP</code>	Nullable. It stays empty until the order is delivered.
99–100	<code>CONSTRAINT</code> <code>chk_delivered_after_order</code> <code>CHECK (delivered_time IS NULL OR delivered_time > order_time)</code>	An order can't be delivered before it was placed. <code>IS NULL</code> OR ... allows orders that aren't delivered yet.
101–102	<code>CONSTRAINT</code> <code>chk_delivered_has_time</code> <code>CHECK (status <> 'Delivered' OR delivered_time IS NOT NULL)</code>	Reads as "if the status is Delivered, there must be a delivered time". Logically, $A \rightarrow B$ is the same as $NOT A OR B$.

`order_item` (lines 105–120)

Line	Code	Explanation
------	------	-------------

105–109	comment	This is the junction table that turns the M : N between orders and dishes into two 1 : M relationships. One order has many dishes, and one dish appears in many orders.
111–113	order_id ... REFERENCES food_order ... ON DELETE CASCADE	CASCADE this time: delete an order and its lines go too. The lines can't exist without their order.
114–116	menu_item_id ... REFERENCES menu_item ... ON DELETE RESTRICT	You can't delete a dish that appears in past orders. Set <code>is_available = FALSE</code> instead.
117	quantity SMALLINT NOT NULL CHECK (quantity BETWEEN 1 AND 50)	SMALLINT (2 bytes) is plenty. The range check blocks 0, negative numbers and typos like 500.
118	unit_price NUMERIC(8,2) NOT NULL CHECK (unit_price > 0)	The price charged at the time of ordering . Menu prices change later, but past bills must stay the same. Practice exercise 32 proves this.
119	PRIMARY KEY (order_id, menu_item_id)	A composite PK : the same dish can appear only once per order. Order more of it by raising quantity instead.

payment (lines 122–136)

Line	Code	Explanation
123	comment	FoodOrder 1 : 1 Payment.

126	payment_id ... PRIMARY KEY	Its own PK.
127– 129	order_id INTEGER NOT NULL UNIQUE REFERENCES food_order ... ON DELETE CASCADE	An FK on its own would allow many payments per order (1 : M). Adding UNIQUE makes it 1 : 1. This is the standard way to build a 1 : 1 relationship.
130– 131	method ... CHECK (method IN ('UPI', 'Card', 'Cash', 'Wallet'))	The payment method.
132– 133	status ... CHECK (status IN ('Pending', 'Paid', 'Refunded', 'Failed'))	The payment status.
134	transaction_ref VARCHAR(40) UNIQUE	The gateway's reference. It's NULL for cash, and unique when present.
135	paid_at TIMESTAMP	When the money was received.

The trigger (lines 138–159)

Why it exists: an FK can check that a dish exists. It **can't** check that the

dish belongs to the *same restaurant as the order*. Copying `restaurant_id` into `order_item` would make that check possible, but it would break 2NF, so a trigger does the check instead.

Line	Code	Explanation
143	<code>CREATE OR REPLACE FUNCTION trg_order_item_same_restaurant()</code>	Defines a function. OR REPLACE lets the file be re-run.
144	<code>RETURNS TRIGGER AS \$\$</code>	A special return type for trigger functions. \$\$ starts the function body (dollar-quoting, so there's no need to escape ' inside).
145	<code>BEGIN</code>	Start of the PL/pgSQL code block.
146	<code>IF (SELECT restaurant_id FROM menu_item WHERE menu_item_id = NEW.menu_item_id)</code>	Looks up which restaurant the dish belongs to. NEW is the row being inserted or updated.

147	IS DISTINCT FROM	Like <>, but NULL-safe. NULL <> 5 gives NULL, which an IF treats as false and would let a bad row through. IS DISTINCT FROM gives TRUE.
148	(SELECT restaurant_id FROM food_order WHERE order_id = NEW.order_id) THEN	Looks up which restaurant the order is from.
149–151	RAISE EXCEPTION 'Menu item % does not belong...', NEW.menu_item_id, NEW.order_id;	If they differ, abort with a clear message. Each % is replaced by the values that follow.
152	END IF;	End of the condition.
153	RETURN NEW;	If the check passes, return the row, which lets the insert go ahead.

154–155	END; \$\$ LANGUAGE plpgsql;	End of the function. It's written in PL/pgSQL, PostgreSQL's procedural language.
157	CREATE TRIGGER order_item_same_restaurant	Attaches the function to the table.
158	BEFORE INSERT OR UPDATE ON order_item	Runs before the row is saved, so a bad row never lands.
159	FOR EACH ROW EXECUTE FUNCTION trg_order_item_same_restaurant();	Runs once for every row affected.

The view (lines 161–174)

Why it exists: order totals are *derived* data. Storing them would risk them drifting out of sync with the lines, which is an update anomaly. A view calculates them fresh every time it's read.

Line	Code	Explanation
165	CREATE VIEW v_order_total AS	A named, saved query. You use it like a table.
166–170	SELECT fo.order_id, fo.restaurant_id, fo.customer_id, fo.status, fo.order_time,	The order columns the reports need. fo is an alias for food_order.

171	<code>SUM(oi.quantity * oi.unit_price)::NUMERIC(10,2) AS order_total</code>	The bill: the sum of quantity × price over all lines. <code>::NUMERIC(10,2)</code> casts the result to 2 decimals.
172– 173	<code>FROM food_order fo JOIN order_item oi ON oi.order_id = fo.order_id</code>	Pairs each order with its lines.
174	<code>GROUP BY fo.order_id;</code>	One row per order. PostgreSQL allows the other <code>fo.*</code> columns without listing them, because they depend on the PK, <code>order_id</code> .

3. `sql/02_indexes.sql`

Line	Code	Explanation
5– 14	comment	Explains the index and why the column order matters .

15–16	<pre>CREATE INDEX idx_restaurant_cuisine_location ON restaurant (cuisine, location);</pre>	The advanced concept. It's a B-tree index sorted by cuisine, then by location, like a phone book sorted by surname then first name. A search for <code>cuisine='X' AND location='Y'</code> jumps straight to the matching entries instead of reading every row.
18–23	comment	PostgreSQL does not automatically index FK columns, unlike MySQL/InnoDB.
24	<pre>CREATE INDEX idx_menu_item_restaurant ON menu_item (restaurant_id);</pre>	Speeds up "all dishes of restaurant X" and the JOINS that use it.
25–27	<pre>... food_order (restaurant_id), (customer_id), (delivery_agent_id)</pre>	Speeds up joins from orders to each parent, and the RESTRICT check when someone tries to delete a parent.
28	<pre>... order_item (menu_item_id)</pre>	<code>order_item.order_id</code> doesn't need its own index, because it's the first column of the composite PK, which is already an index. <code>menu_item_id</code> is second, so it gets one.

The leftmost-prefix rule: the index on `(cuisine, location)` helps searches on `cuisine` alone and on `cuisine + location`. It can't seek on `location` alone, for the same reason a phone book sorted by surname can't find everyone named "Rahul" quickly.

4. sql/03_seed_data.sql

Lines	What	Explanation
1–5	header	Row counts per table.
7	BEGIN;	Starts a transaction . Either <i>all</i> the data loads or <i>none</i> of it does. If any insert fails, you never end up with half a database.
9–24	restaurants	INSERT INTO restaurant (col1, col2, ...) VALUES (...), (...), ...; is a multi-row insert, one statement for 11 rows. Explicit IDs (1–11) make the FKs below readable. In row 1, (1, 'Punjab Da Dhaba', 'North Indian', 'Kharghar', 'Shop 4...', '02227741001', 4.4), the values match the column list in order. Rows 1 and 7 share <i>North Indian + Kharghar</i> on purpose, so the index search returns 2 rows.
26–93	menu items	53 rows. Restaurant <i>r</i> owns dishes 5r–4 to 5r, so restaurant 2 owns 6–10. -- lines are comments that group them by restaurant. FALSE/TRUE is is_veg. Restaurant 11 (Malvani Tadka) has dishes but will get no orders , so the LEFT JOIN queries show a ₹0 row.
95–113	customers	15 rows. They use @example.com emails, a domain reserved for examples, so no real person is ever emailed.
115–126	delivery agents	8 rows. Line 123 contains 'Joseph D' 'Souza': two single quotes is how you write an apostrophe inside an SQL string. Line 125 has NULL for the bicycle's vehicle number.

128–194	orders	See the breakdown below.
196–262	order lines	See the breakdown below.
264–295	payments	See the breakdown below.
297–306	setval	See the breakdown below.
308	COMMIT;	Saves everything permanently, in one go.

Orders (lines 134–194): INSERT ... SELECT ... FROM (VALUES ...) JOIN

```
INSERT INTO food_order (order_id, customer_id, ..., deliver
SELECT v.oid, v.cid, v.rid, v.aid,
      v.t::TIMESTAMP,
      c.address,
      v.st,
      CASE WHEN v.mins IS NULL THEN NULL
           ELSE v.t::TIMESTAMP + make_interval(mins => v
FROM (VALUES
  ( 1,  1,  1,  1, '2026-09-02 13:05', 'Delivered', 34),
  ...
) AS v(oid, cid, rid, aid, t, st, mins)
JOIN customer c ON c.customer_id = v.cid;
```

Why this pattern: `delivery_address` is copied from the customer **by the database**, not retyped, so there are no typos. The delivered time is *calculated* from minutes, so it's always after the order time and never trips the CHECK constraint. Line 159 (order 16) and the line for order 47 are Cancelled with NULL minutes. Orders 48–49 are still in progress.

Order lines (lines 199–262)

```
INSERT INTO order_item (order_id, menu_item_id, quantity,
SELECT v.oid, v.mid, v.qty, m.price           -- 200 price
FROM (VALUES ( 1, 1,1),( 1, 4,4), ... )      -- each (ord
AS v(oid, mid, qty)
JOIN menu_item m ON m.menu_item_id = v.mid;  -- 262 look
```

(1, 4,4) means "order 1 contains dish 4 (Butter Naan) × 4". The **Dragon Wok** block (orders 12–16) is built so that *Veg Hakka Noodles* and *Chilli Paneer* tie at 3 orders each. That tests whether Q1 handles ties. Every line also passes through the trigger, so a wrong restaurant would make the whole load fail.

Payments (lines 267–295): three CASE expressions

Lines	Column	Rule
270–274	status	Cancelled order → Refunded. Cash and not yet delivered → Pending. Anything else → Paid.
275–277	transaction_ref	Cash → NULL. Otherwise 'QB' + date + 5-digit order number, e.g. QB2026090200001. joins strings, and lpad(..., 5, '0') pads the number with zeros.
278–281	paid_at	Cash is paid when delivered. Online payments are made 1 minute after ordering.

Resetting the ID counters (lines 297–306)

```
SELECT setval(pg_get_serial_sequence('restaurant','restau
```

Because we inserted explicit IDs, the auto-numbering counter is still at 1. Without this line, the next *app* insert would try ID 1 and crash with a duplicate-key error. `pg_get_serial_sequence` finds the counter behind the column, and `setval` moves it to the current maximum.

order_item has no line here, because it has no identity column.

5. sql/04_queries.sql

Lines 1–6 set one business rule for every query: *cancelled orders aren't demand, and only delivered orders count as revenue*. Each query is preceded by \echo lines. These are **psql commands**, not SQL. They print a banner into the output file.

Q1. Most popular dish per restaurant (lines 8–37), the query the brief requires

Line	Code	Explanation
14–18	SELECT r.restaurant_id, r.name AS restaurant, m.name AS most_popular_dish, COUNT(*) AS times_ordered, SUM(oi.quantity) AS units_sold	COUNT (*) = how many orders included this dish. SUM(quantity) = how many plates were sold. AS renames the column in the output.
19	FROM order_item oi	Start from the order lines, where "dish was ordered" facts live.
20	JOIN menu_item m ON m.menu_item_id = oi.menu_item_id	The required JOIN between order lines and MenuItem, to get the dish name and its restaurant.
21	JOIN food_order fo ON fo.order_id = oi.order_id	Needed to read the order's status.

22	JOIN restaurant r ON r.restaurant_id = m.restaurant_id	Gets the restaurant's name.
23	WHERE fo.status <> 'Cancelled'	Drops cancelled orders <i>before</i> counting.
24	GROUP BY r.restaurant_id, r.name, m.menu_item_id, m.name	One group per (restaurant, dish). Every non-aggregated column in SELECT must appear here.
25	HAVING COUNT(*) = (	Keeps only groups whose count equals the value from the subquery below. HAVING filters <i>groups</i> , while WHERE filters <i>rows</i> .
28	SELECT MAX(dish_count)	The ranking subquery. It finds the highest dish count...
29– 35	FROM (SELECT COUNT(*) AS dish_count FROM order_item oi2 JOIN ... WHERE m2.restaurant_id = r.restaurant_id AND ... GROUP BY oi2.menu_item_id) AS per_dish	...among the dishes of this restaurant only . m2.restaurant_id = r.restaurant_id refers to r from the <i>outer</i> query, which makes this a correlated subquery : it re-runs once per outer group. The inner aliases (oi2, m2, fo2) must differ from the outer ones.
37	ORDER BY r.restaurant_id, m.name;	Sorts the output.

Tie handling: if two dishes both equal the max, *both* pass the HAVING, which is why Dragon Wok appears twice.

Q1b. The same result with window functions (lines 40–67)

Line	Code	Explanation
45–54	WITH dish_stats AS (SELECT ... GROUP BY ...)	A CTE (Common Table Expression), a named temporary result. It holds counts and units per dish, the same numbers as Q1.
55–61	ranked AS (SELECT ds.*, RANK() OVER (...), ROW_NUMBER() OVER (...))	A second CTE that builds on the first.
57	RANK() OVER (PARTITION BY restaurant_id ORDER BY times_ordered DESC)	A window function . PARTITION BY restarts the ranking for each restaurant, and ORDER BY ... DESC gives the highest count rank 1. Ties share a rank (1, 1, 3...). Unlike GROUP BY, a window function keeps every row.
58–59	ROW_NUMBER() OVER (... ORDER BY times_ordered DESC, units_sold DESC, dish)	Always gives 1, 2, 3, with no ties. Ties are broken by units sold, then alphabetically. This picks exactly one dish to promote.
62–63	CASE WHEN promo_pick = 1 THEN 'YES' ELSE ' ' END AS promote	Marks the chosen dish.
65	JOIN restaurant r USING (restaurant_id)	USING is shorthand for ON ranked.restaurant_id = r.restaurant_id when the column names match.
66	WHERE popularity_rank = 1	Keeps only the top dishes. You can't filter on a window function in the same SELECT where it's computed, which is why the CTE is needed.

Q2. Revenue per restaurant (lines 70–86), required outcome

Line	Code	Explanation
78	COUNT(t.order_id) AS delivered_orders	COUNT(column) skips NULLs, so a restaurant with no orders gets 0, not 1. COUNT(*) would give 1.
79	COALESCE(SUM(t.order_total), 0) AS total_revenue	GROUP BY + SUM on order totals , as the brief asks. SUM of nothing is NULL, and COALESCE(x, 0) turns NULL into 0.
80	COALESCE(ROUND(AVG(t.order_total), 2), 0)	Average bill, rounded to 2 decimals.

81–84	FROM restaurant r LEFT JOIN v_order_total t ON t.restaurant_id = r.restaurant_id AND t.status = 'Delivered'	LEFT JOIN keeps every restaurant, even ones with no matching orders. ⚠ The status filter is in ON, not in WHERE. In WHERE, it would throw away the NULL rows and silently turn the LEFT JOIN into an INNER JOIN, and Malvani Tadka would disappear.
85	GROUP BY r.restaurant_id, r.name, r.cuisine	One row per restaurant.
86	ORDER BY total_revenue DESC	Highest revenue first.

Q2b (lines 89–101)

The same revenue, calculated straight from `order_item` (`SUM(oi.quantity * oi.unit_price)`) without the view. This proves the view is correct. It uses an **INNER JOIN** and the filter is in `WHERE`, so restaurants with no sales drop out (10 rows instead of 11). That's a good example of the LEFT vs INNER difference.

Q3. Search by cuisine and area (lines 104–114)

`WHERE cuisine = 'North Indian' AND location = 'Kharghar' AND is_active` is exactly the search the index was built for. `AND is_active` is shorthand for `is_active = TRUE`. `ORDER BY rating DESC` puts the best-rated first.

Q4. Above-average restaurants (lines 117–133)

Line	Explanation
122–126	Revenue per restaurant, as in Q2 but with an INNER JOIN.
127	HAVING SUM(t.order_total) > (compares each restaurant's total...
128–132	...with AVG (rest_rev) taken over a derived table (a subquery in FROM, aliased x) that holds one revenue figure per restaurant. You need two levels because you can't nest aggregates, so AVG (SUM(...)) isn't allowed.

Q5. Dishes never ordered (lines 136–150)

Line	Explanation
141–143	Every dish, joined to its restaurant.
144	WHERE NOT EXISTS (keeps the dish only if the subquery finds nothing .
145	SELECT 1. With EXISTS, the selected value doesn't matter, only whether any row exists. 1 is the convention.
146–149	Looks for any non-cancelled order line for this dish. It's correlated through oi.menu_item_id = m.menu_item_id.

NOT EXISTS is safer than NOT IN: if the NOT IN list contains a NULL, NOT IN returns no rows at all.

Q6. Delivery-agent performance (lines 153–169)

Line	Explanation
------	-------------

162	<code>fo.delivered_time - fo.order_time</code> gives an <i>interval</i> . <code>EXTRACT(EPOCH FROM ...)</code> converts it to seconds, <code>/ 60</code> to minutes, <code>AVG</code> averages them, and <code>ROUND(..., 1)</code> keeps 1 decimal.
165–167	<code>LEFT JOIN</code> with the status filter in <code>ON</code> , the same pattern as Q2, so an agent with 0 deliveries would still appear.
169	Sorts by deliveries, then speed.

Q7. Top 5 customers (lines 172–186)

Customer joined to `v_order_total`, delivered orders only, `GROUP BY` customer, `SUM` of spend, `ORDER BY ... DESC`, and `LIMIT 5` keeps the first 5 rows.

Q8. Revenue by cuisine (lines 189–201)

`COUNT(DISTINCT r.restaurant_id)` counts restaurants per cuisine **without duplicates**. A plain `COUNT` would count one per order row. `LEFT JOIN + COALESCE` keeps Malvani at 0.

Q9. Invoice for order #26 (lines 204–226)

A **7-table JOIN**: `food_order → customer, restaurant, delivery_agent, order_item → menu_item, payment`. Line 209: `to_char(fo.order_time, 'DD-Mon-YYYY HH24:MI')` formats the date as 21-Sep-2026 20:45. Line 216 calculates each line total. Line 225 picks one order.

Q10. Payment-method split (lines 229–241)

Line 236, `ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER (), 1), 1)`, is a window function **over an aggregate**. `COUNT(*)` gives this method's orders. `SUM(COUNT(*) OVER ())` adds up all the groups' counts. An empty `OVER ()` means the window is the whole result.

Writing `100.0` instead of `100` forces decimal division. Integer division would give $7/45 = 0$.

6. `sql/05_index_demo.sql`

Line	Code	Explanation
9	<code>BEGIN;</code>	Everything below is temporary, and gets undone at line 67.
11	<code>INSERT INTO restaurant (name, cuisine, ...)</code>	No <code>restaurant_id</code> is given, so <code>IDENTITY</code> numbers the rows automatically.
12	<code>'Test Kitchen ' g</code>	Builds a unique name from the counter <code>g</code> .
13–15	<code>(ARRAY['North Indian', ..., 'Thai'])[1 + g % 12]</code>	Picks one of 12 cuisines in turn. <code>g % 12</code> gives 0–11. Arrays in PostgreSQL start at 1, hence <code>1 +</code> .

16– 19	<code>(ARRAY['Kharghar', ...])[1 + (g / 12) % 20]</code>	Picks one of 20 areas. <code>g / 12</code> is integer division, which shifts the pattern so every cuisine × area combination (240 in total) is used.
20– 21	address and phone	Unique dummy values. <code>lpad(g : TEXT, 10, '0')</code> turns 7 into 0000000007, so phones stay unique and satisfy the UNIQUE constraint.
22	<code>round((3 + random() * 2)::NUMERIC, 1)</code>	A random rating from 3.0 to 5.0.
23	<code>FROM generate_series(1, 200000) AS g;</code>	Produces the numbers 1 to 200,000, one row each, so 200,000 inserts happen in a single statement.
25	<code>ANALYZE restaurant;</code>	Refreshes the planner's statistics, so it knows the table is now big.
27– 29	<code>\echo + SELECT COUNT(*)</code>	Shows the catalogue size: 200,011.

34	<pre>DROP INDEX idx_restaurant_cuisine_location;</pre>	Removes the index, inside the transaction only, to measure the slow path.
36– 39	<pre>EXPLAIN (ANALYZE, COSTS OFF, SUMMARY ON) SELECT ...</pre>	EXPLAIN shows the <i>plan</i>. ANALYZE actually runs the query and reports real times. COSTS OFF hides estimated costs to keep the output readable. SUMMARY ON prints the total execution time. Result: Seq Scan, which reads all 200k rows.
44– 45	<pre>CREATE INDEX ...; ANALYZE ...;</pre>	Puts the index back.
47– 50	the same EXPLAIN	Result: Bitmap Index Scan , which touches only the 835 matching rows. It's many times faster.
54– 56	<pre>EXPLAIN ... WHERE cuisine = 'Italian'</pre>	Leftmost prefix: the first column alone still uses the index.

59– 65	<code>EXPLAIN (ANALYZE ...) ... WHERE location = 'Powai'</code>	Second column alone: PostgreSQL ≤ 17 can't use the index. PostgreSQL 18 can, through <i>skip scan</i> : it jumps to each distinct cuisine and looks for Powai inside it. The output shows "Index Searches: 25". It works, but it costs more than a direct seek.
67	<code>ROLLBACK;</code>	Undoes everything since BEGIN: the 200k rows are gone and the index is back to its original state. The real sample data is untouched.

7. `sql/06_integrity_tests.sql`

Line	Code	Explanation
7	<code>DO \$\$</code>	Runs an anonymous code block , i.e. procedural code without creating a function.

8	DECLARE	Variable declarations follow.
9	tests TEXT[][] := ARRAY[	A 2-D text array. Each entry is [description, SQL to try].
10– 12	['Order for a customer that does not exist (FK)', \$q\$INSERT ... VALUES (999, 1, 1, 'Nowhere')\$q\$]	Customer 999 doesn't exist, so the FK should reject this. \$q\$... \$q\$ is a <i>named dollar quote</i> , used so the SQL inside can contain ' and won't clash with the outer \$\$.
13– 15	missing restaurant	Restaurant 999. Expect an FK error.
16– 18	NULL delivery agent	Expect a NOT NULL error.
19– 21	dish 21 (Biryani Darbar) on order 1 (Punjab Da Dhaba)	Expect the trigger to fire.
22– 23	DELETE FROM restaurant WHERE restaurant_id = 1	Expect RESTRICT. Note that the menu_item FK is checked first and blocks the delete, before the food_order FK is reached. Either way, the restaurant is protected.
24– 25	second payment for order 1	Expect the UNIQUE (1 : 1) error.
26– 28	price –10	Expect a CHECK error.
29– 30	status 'Lost'	Expect a CHECK error.
31– 33	a duplicate email	Expect a UNIQUE error.
35	i INT;	The loop counter.

37	<code>FOR i IN 1 .. array_length(tests, 1) LOOP</code>	Loops over the 9 tests. <code>array_length(tests, 1)</code> is the size of the first dimension.
38	<code>BEGIN</code>	An inner block . An inner block with an <code>EXCEPTION</code> clause gets its own <i>savepoint</i> , so a failure here is undone without killing the loop.
39	<code>EXECUTE tests[i][2];</code>	Runs the SQL text. This is dynamic SQL.
40	<code>RAISE NOTICE 'FAIL ... was accepted!'</code>	Only reached if the bad statement succeeded , which would mean a missing constraint.
41	<code>EXCEPTION WHEN OTHERS THEN</code>	Catches any error.
42– 43	<code>RAISE NOTICE 'OK %' ...; RAISE NOTICE 'rejected: %', SQLERRM;</code>	Prints a pass, plus the database's own error message. <code>SQLERRM</code> holds the text of the caught error.
44– 46	<code>END; END LOOP; END \$ \$;</code>	Close the inner block, the loop and the DO block.

Because each failing statement is rolled back to its savepoint, **nothing changes in the database**. The final output is nine `OK` lines.

Concepts index

Concept	Where it's used
Primary key (single / composite)	every table · <code>order_item</code> line 119

Foreign key + ON DELETE RESTRICT / CASCADE	schema lines 34–36, 84–92, 111–116, 127–129
NOT NULL, UNIQUE, CHECK	throughout 01_schema.sql
1 : 1 via FK + UNIQUE	payment.order_id, line 127
M : N via junction table	order_item, lines 110–120
Trigger	schema lines 143–159
View	schema lines 165–174
Composite index, leftmost prefix	02_indexes.sql lines 15–16 · demo lines 54–65
Transactions (BEGIN / COMMIT / ROLLBACK)	seed lines 7 + 308 · demo lines 9 + 67
INNER vs LEFT JOIN	Q2 vs Q2b
GROUP BY, HAVING	Q1, Q4
Correlated subquery	Q1 lines 25–36 · Q5
Derived table	Q1 line 29 · Q4 line 129
EXISTS / NOT EXISTS	Q5
CTE (WITH)	Q1b
Window functions	Q1b (RANK, ROW_NUMBER) · Q10 (SUM OVER)
CASE	Q1b line 63 · seed lines 140, 270–281
EXPLAIN ANALYZE	05_index_demo.sql
Exception handling, dynamic SQL	06_integrity_tests.sql